

ADP470S

Applications Development Practice 4

Comprehensive Test 1 Study Guide — 2026

Cape Peninsula University of Technology
Faculty of Informatics & Design | NQF Level 7

What this guide covers (T1 Syllabus)

- SO1 — What is an Algorithm? Algorithm analysis and comparison
- SO2 — Recursion vs Iteration: Factorial and Fibonacci
- SO3 — Algorithm Complexity: Linear Search and Binary Search
- SO4 — Bubble Sort: best-case, worst-case, and average-case complexity
- SO5 — 2D / Multidimensional Arrays; Introduction to Linked Lists
- SO6 — Doubly Linked Lists and Circular Linked Lists

Recommended Textbook

A Common-Sense Guide to Data Structures and Algorithms by Jay Wengrow is an excellent choice for T1. It explains Big O notation, arrays, and sorting algorithms using plain English and visual step-by-step traces. Its coverage of $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, linear search, binary search, bubble sort and insertion sort aligns directly with your T1 outcomes. It will not cover every topic in depth (e.g. linked lists get moderate coverage) but it is more than sufficient to build solid understanding for the test. Use it alongside these notes and the lecturer slides.

1. What is an Algorithm?

Definition

An algorithm is a finite, definite procedure for solving a problem.

- Finite — it must terminate in a finite number of steps
- Definite — at every step it makes measurable progress; no guessing

Think of an algorithm as a recipe. A recipe has a clear list of ingredients (inputs), a step-by-step method that always finishes, and a dish at the end (output). Every step is specific — "bake at 180 °C for 30 minutes" is definite; "bake until it looks done" is not.

1.1 A Concrete Example — Euclid's HCF Algorithm

The Highest Common Factor (HCF) of two positive integers can be found with Euclid's algorithm, one of the oldest known algorithms ($\approx$ 300 BC):

```
algorithm hcf(n, m):           // find HCF of positive integers n and m
  repeat:
    l = remainder of (n ÷ m)    // modulo operation
    n = m
    m = l
  until m = 0
  return n                     // n now holds the HCF
```

Example — HCF(48, 18):

```
Step 1: l = 48 mod 18 = 12 → n=18, m=12
Step 2: l = 18 mod 12 = 6  → n=12, m=6
Step 3: l = 12 mod 6  = 0  → n=6,  m=0 → STOP
HCF = 6  ✓
```

Why Euclid's algorithm is considered good

- Simple to understand and implement
- Guaranteed to terminate (m always decreases each step)
- Much faster than the brute-force alternative of counting down from $\min(n, m)$
- For large numbers the difference in speed is enormous

1.2 What Makes an Algorithm Good?

Two primary qualities determine algorithm quality:

- Speed (Time Complexity) — how many operations does it perform as input size grows?
- Space (Space Complexity) — how much extra memory does it require?

These qualities are measured using Big O notation, covered in the next section.

2. Big O Notation — Algorithm Complexity

What Big O tells us

Big O notation describes how the time or space requirement of an algorithm scales as the input size n grows large. It always describes the worst-case scenario and ignores constant factors and lower-order terms.

Notation	Name	Meaning
$O(1)$	Constant	Execution time is the same regardless of input size.
$O(\log n)$	Logarithmic	Execution time grows very slowly — doubles input $\rightarrow$ +1 step. E.g. Binary Search.
$O(n)$	Linear	Execution time grows in direct proportion to input size. E.g. Linear Search.
$O(n \log n)$	Linearithmic	Common in efficient sorts like QuickSort and MergeSort.
$O(n^2)$	Quadratic	Execution time grows with the square of input size. Common with nested loops. E.g. Bubble Sort.
$O(2^n)$	Exponential	Execution time doubles with every extra element. Avoid for large inputs.

2.1 $O(1)$ — Constant Time

No matter how large the input, the algorithm always takes the same time.

```
// Check if the first element is null — always ONE operation
boolean isFirstNull(String[] arr) {
    if (arr[0] == null) return true;
    return false;
}
```

2.2 $O(n)$ — Linear Time

As input size n doubles, the number of operations doubles. Classic example: Linear Search.

```
// Linear search — worst case: scan every element
boolean contains(String[] arr, String value) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i].equals(value)) return true;
    }
    return false; // worst case: checked all n elements
}
```

Even though the function may return early, Big O always measures the worst case — when the value is not found and every element is checked.

2.3 $O(n^2)$ — Quadratic Time

Two nested loops each running n times = $n \times n = n^2$ operations. Classic example: Bubble Sort and checking for duplicates.

```
boolean hasDuplicates(String[] arr) {
    for (int i = 0; i < arr.length; i++) {
        for (int j = 0; j < arr.length; j++) {
```

```

        if (i != j && arr[i].equals(arr[j])) return true;
    }
    return false;
}

```

2.4 $O(\log n)$ — Logarithmic Time

Each step halves the problem. For $n=1\,000\,000$ only about 20 steps are needed. Classic example: Binary Search on a sorted array.

```

int binarySearch(int[] a, int value, int left, int right) {
    while (left <= right) {
        int mid = (left + right) / 2;
        if (a[mid] == value) return mid;           // found
        else if (a[mid] > value) right = mid - 1;  // search left half
        else left = mid + 1;                       // search right half
    }
    return -1; // not found
}

```

Why Binary Search is $O(\log n)$

Each comparison eliminates half the remaining elements. Starting with $n=16$:

Step 1: $16 \rightarrow 8$ elements remain
 Step 2: $8 \rightarrow 4$ elements remain
 Step 3: $4 \rightarrow 2$ elements remain
 Step 4: $2 \rightarrow 1$ element remains

Only 4 steps for 16 elements. In general: $\text{steps} = \log_2(n)$. For $n=1\,024 \rightarrow$ only 10 steps!

2.5 $O(2^n)$ — Exponential Time

The number of operations doubles for every extra element. Recursive Fibonacci without memoisation is a famous example. $n=50$ would require over 1 trillion operations.

2.6 Growth Rate Summary

Quick comparison: number of operations for $n = 1000$

$O(1)$	=	1	operation
$O(\log n)$	≈	10	operations
$O(n)$	=	1 000	operations
$O(n \log n)$	≈	10 000	operations
$O(n^2)$	=	1 000 000	operations
$O(2^n)$	=	$2^{1000} \approx 10^{301}$	(impossible!)

3. Linear Search and Binary Search

3.1 Linear Search — $O(n)$

Examine each element one by one from the start until you find the target or exhaust the array.

```
int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i; // found at index i
    }
    return -1; // not found
}
```

Case	Scenario	Complexity
Best Case	Target is first element	$O(1)$
Average Case	Target is somewhere in the middle	$O(n/2) \rightarrow O(n)$
Worst Case	Target not found (scan all n elements)	$O(n)$

3.2 Binary Search — $O(\log n)$

Requires a sorted array. Compare the target with the middle element, then eliminate the half that cannot contain the target.

Step-by-step trace — find 7 in [1, 3, 5, 7, 9, 11, 13]

Array (indices): [0]=1 [1]=3 [2]=5 [3]=7 [4]=9 [5]=11 [6]=13

Step 1: left=0, right=6 → mid=3 → a[3]=7 → FOUND at index 3!

Trace for finding 11:

Step 1: left=0, right=6 → mid=3 → a[3]=7 < 11 → search RIGHT half

Step 2: left=4, right=6 → mid=5 → a[5]=11 = 11 → FOUND!

```
// Iterative Binary Search
int binarySearch(int[] a, int value, int left, int right) {
    while (left <= right) {
        int mid = (left + right) / 2;
        if (a[mid] == value) return mid;
        else if (a[mid] > value) right = mid - 1; // go left
        else left = mid + 1; // go right
    }
    return -1;
}

// Recursive Binary Search
int binarySearch(int[] a, int value, int left, int right) {
    if (left > right) return -1;
    int mid = (left + right) / 2;
    if (a[mid] == value) return mid;
    else if (a[mid] > value) return binarySearch(a, value, left, mid - 1);
    else return binarySearch(a, value, mid + 1, right);
}
```

Key requirement — sorted array

Binary Search only works on a sorted array. Sorting an unsorted array first ($O(n \log n)$) and then searching ($O(\log n)$) is still far better than linear search $O(n)$ when you need to search many times.

4. Recursion vs Iteration

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a smaller version of the same problem. Every recursive function must have:

- A base case — the stopping condition (prevents infinite recursion)
- A recursive case — reduces the problem toward the base case

The runtime uses a call stack to manage recursive calls. Each call creates a new stack frame that stores the function parameters, local variables, and return address. When the base case is reached, frames are removed one by one. Too many calls cause a Stack Overflow.

4.1 Factorial

Mathematical definition: $n! = n \times (n-1) \times (n-2) \times \dots \times 1$ and $0! = 1$

```
// Recursive Factorial (Java)
public static int factorial(int n) {
    if (n == 0) return 1;          // base case
    return n * factorial(n - 1);  // recursive case
}

// Iterative Factorial (Java)
public static int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Call Stack Trace — factorial(5)

Expansion (going DOWN the stack):

```
factorial(5) → 5 × factorial(4)
factorial(4) → 4 × factorial(3)
factorial(3) → 3 × factorial(2)
factorial(2) → 2 × factorial(1)
factorial(1) → 1 × factorial(0)
factorial(0) → 1 ← BASE CASE
```

Unwinding (going UP the stack):

```
factorial(1) = 1 × 1 = 1
factorial(2) = 2 × 1 = 2
factorial(3) = 3 × 2 = 6
factorial(4) = 4 × 6 = 24
factorial(5) = 5 × 24 = 120 ✓
```

4.2 Fibonacci

The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Definition: $F(0)=0$, $F(1)=1$, $F(n) = F(n-1) + F(n-2)$ for $n > 1$

```
// Recursive Fibonacci (Java) - simple but SLOW for large n
public static int fibonacci(int n) {
    if (n <= 1) return n;           // base case
    return fibonacci(n-1) + fibonacci(n-2); // two recursive calls!
}

// Iterative Fibonacci (Java) - efficient
public static int fibonacci(int n) {
    if (n <= 1) return n;
    int prev = 0, curr = 1;
    for (int i = 2; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```

Why recursive Fibonacci is inefficient — $O(2^n)$

Recursive Fibonacci calls itself TWICE per call. For fibonacci(5):

fib(5) calls fib(4) and fib(3)

fib(4) calls fib(3) and fib(2) ← fib(3) computed AGAIN

fib(3) calls fib(2) and fib(1) ← fib(2) computed multiple times

The same sub-problems are recomputed repeatedly. For $n=50$, over $2^{50} \approx 1$ trillion calls would be made. The iterative version solves this in just 50 steps — $O(n)$.

4.3 Complexity Comparison

	Time (Rec)	Space (Rec)	Time (Iter)	Space (Iter)
Factorial	$O(n)$	$O(n)$ stack	$O(n)$	$O(1)$
Fibonacci	$O(2^n)$ ⚠	$O(n)$ stack	$O(n)$	$O(1)$

Key takeaway

For factorial, recursive and iterative have the same time complexity $O(n)$ but the iterative version uses less space $O(1)$ vs $O(n)$. For Fibonacci, the iterative version is dramatically better: $O(n)$ vs $O(2^n)$. This is a key exam point.

5. Bubble Sort

How Bubble Sort works

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element "bubbles" to its correct position at the end.

For an array of n numbers:

- Each pass requires $(n-1)$ comparisons
- There are $(n-1)$ passes
- Total comparisons: $(n-1) \times (n-1) \approx n^2$ — this is why Bubble Sort is $O(n^2)$

```
public void sort(int[] a) {
    for (int j = 0; j < a.length - 1; j++) {           // outer: n-1 passes
        for (int i = 0; i < a.length - 1 - j; i++) {    // inner: shrinks each
pass
            if (a[i] > a[i+1]) {
                int temp = a[i];    // swap
                a[i] = a[i+1];
                a[i+1] = temp;
            }
        }
    }
}
```

5.1 Step-by-Step Trace — Sort [5, 3, 2, 1, 4]

Bubble Sort Trace (ascending order)

Initial: [5, 3, 2, 1, 4]

Pass 1:

Compare 5,3 → swap → [3, 5, 2, 1, 4]
 Compare 5,2 → swap → [3, 2, 5, 1, 4]
 Compare 5,1 → swap → [3, 2, 1, 5, 4]
 Compare 5,4 → swap → [3, 2, 1, 4, 5] ← 5 is in place

Pass 2:

Compare 3,2 → swap → [2, 3, 1, 4, 5]
 Compare 3,1 → swap → [2, 1, 3, 4, 5]
 Compare 3,4 → no swap → [2, 1, 3, 4, 5] ← 4,5 in place

Pass 3:

Compare 2,1 → swap → [1, 2, 3, 4, 5]
 Compare 2,3 → no swap → [1, 2, 3, 4, 5] ← 3,4,5 in place

Pass 4:

Compare 1,2 → no swap → [1, 2, 3, 4, 5] SORTED! ✓

5.2 Best, Worst and Average Case

Case	Input Condition	Comparisons	Big O
Best Case	Already sorted (no swaps needed)	$n-1$ (one pass, no swaps)	$O(n) *$
Average Case	Random order	$\approx n^2/2$	$O(n^2)$
Worst Case	Reverse sorted	$(n-1)^2 \approx n^2$	$O(n^2)$

* Best case $O(n)$ requires the optimised version that tracks whether any swap occurred in a pass and terminates early.

5.3 Properties of Bubble Sort

- Stable — equal elements retain their original relative order
- In-place — only $O(1)$ extra space needed for the temp variable
- Adaptive — $O(n)$ when data is nearly sorted (with early-exit optimisation)
- Simple to implement but inefficient for large datasets

5.4 Other Sorting Algorithms (Overview)

The following sorts are also in the syllabus. You should know their complexity.

Algorithm	Best	Worst	Key Idea
Bubble Sort	$O(n)$	$O(n^2)$	Swap adjacent pairs; largest bubbles to end
Selection Sort	$O(n^2)$	$O(n^2)$	Find minimum, place it at front of unsorted region
Insertion Sort	$O(n)$	$O(n^2)$	Insert each element into its correct position in sorted region
Shell Sort	$O(n \log n)$	$O(n^{1.2})$	Insertion sort with diminishing gap increments (Knuth)
Quick Sort	$O(n \log n)$	$O(n^2)$	Partition around pivot; recurse on each half
Merge Sort	$O(n \log n)$	$O(n \log n)$	Divide in half, sort each, merge — always $O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	Build max-heap; repeatedly extract max

6. Arrays — 1D, 2D and Multidimensional

What is an Array?

An array is a fixed-size, indexed collection of elements of the same type stored in contiguous memory. Elements are accessed in $O(1)$ time using their index.

6.1 1D Array

```
int[] numbers = {10, 20, 30, 40, 50};
// Access element at index 2:
System.out.println(numbers[2]); // prints 30
// Index runs from 0 to length-1
```

6.2 2D Array (Matrix)

A 2D array is an array of arrays — think of it as a table with rows and columns.

```
// Declare and initialise a 3x4 matrix (3 rows, 4 columns)
int[][] matrix = {
    {1, 2, 3, 4}, // row 0
    {5, 6, 7, 8}, // row 1
    {9, 10, 11, 12} // row 2
};

// Access element at row 1, column 2:
System.out.println(matrix[1][2]); // prints 7

// Traverse entire matrix:
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

6.3 Matrix Multiplication — $O(n^3)$

To multiply two $n \times n$ matrices, three nested loops are required — one over rows, one over columns, one over the shared dimension. This is $O(n^3)$.

```
int[][] multiply(int[][] A, int[][] B, int n) {
    int[][] C = new int[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    return C;
}
```

6.4 Limitations of Arrays

- Fixed size — you must declare the size upfront; resizing requires creating a new array and copying all elements
- Insertion / deletion in the middle is $O(n)$ — all subsequent elements must shift
- Wasted memory if the array is oversized

- **These limitations motivate the use of Linked Lists**

7. Linked Lists

What is a Linked List?

A Linked List is a dynamic data structure where each element (called a node) contains data and a reference (pointer) to the next node. Unlike arrays, nodes do not need to be stored in contiguous memory.

Each node has two parts:

- Data — the value stored
- Next — a reference to the next node in the list (null for the last node)

Visual representation of a Singly Linked List



```
// Node class (Java)
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// LinkedList with head pointer
class LinkedList {
    Node head;
}
```

7.1 Singly Linked List — Operations & Complexity

Operation	Complexity	Notes
Insert at head	$O(1)$	Update head pointer — very fast
Insert at tail	$O(n)$	Must traverse to last node first
Delete at head	$O(1)$	Update head pointer — very fast
Delete at tail / middle	$O(n)$	Must traverse to find node before target
Search	$O(n)$	No random access; must scan from head
Access by index	$O(n)$	No random access — unlike arrays

7.2 Limitations of a Singly Linked List

- Can only traverse forward — you cannot go back to a previous node
- To delete a node you need a reference to the PREVIOUS node, requiring a full traversal

- No tail pointer by default — appending to the end is $O(n)$
- Each node uses extra memory for the next pointer

7.3 Doubly Linked List

Each node has two pointers: one to the next node and one to the previous node. This solves the backward-traversal problem.

Visual — Doubly Linked List

```

head                                     tail
|                                     |
[ / | 5 | • ] ↔ [ • | 3 | • ] ↔ [ • | 2 | • ] ↔ [ • | 1 | • ] ↔ [ • | 4 | / ]
/ = null pointer    • | data | •    format: [prev | data | next]

```

```

class DNode {
    int data;
    DNode next;
    DNode prev; // extra pointer — points backward
    DNode(int data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}

```

Advantages of Doubly Linked List

- Can traverse in both directions — forward and backward
- Deletion of a known node is $O(1)$ — no need to find previous node
- Easier to implement operations at both ends

Disadvantages of Doubly Linked List

- Each node uses more memory — two pointers instead of one
- More complex to implement (must update both prev and next on every insert/delete)

7.4 Circular Linked List

The last node's next pointer links back to the head (for singly circular) or the head's prev links to tail (for doubly circular). There is no null terminator.

Visual — Singly Circular Linked List

```

head
|
[ 5 | • ] → [ 3 | • ] → [ 2 | • ] → [ 4 | • ] → ↑
↑ _____ ↑ (last node points back to head)

```

Advantages of Circular Linked List

- No null pointer — traversal can continue indefinitely from any node

- Useful for round-robin scheduling (operating systems)
- Any node can act as the starting point
- Appending to end and accessing head from tail is efficient

7.5 Comparison: Arrays vs Linked Lists

Feature	Array	Linked List
Size	Fixed at declaration	Dynamic — grows/shrinks
Memory	Contiguous (cache-friendly)	Non-contiguous (pointers)
Access by index	$O(1)$ — direct access	$O(n)$ — must traverse
Insert/Delete at head	$O(n)$ — shift elements	$O(1)$ — update pointer
Insert/Delete at middle	$O(n)$	$O(n)$ traverse + $O(1)$ splice
Memory overhead	Low — just data	Higher — pointer(s) per node
Search	$O(n)$ unsorted / $O(\log n)$ sorted	$O(n)$

8. Quick Revision Summary

Use this section for last-minute review before your test.

Algorithms — Key Points

- An algorithm is finite (terminates) and definite (no guessing)
- Quality is measured by time complexity and space complexity
- Big O describes worst-case asymptotic behaviour
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

Recursion vs Iteration — Key Points

- Recursion uses the runtime call stack; too deep → StackOverflow
- Every recursive function needs a base case
- Recursive factorial: $O(n)$ time, $O(n)$ space
- Iterative factorial: $O(n)$ time, $O(1)$ space — more memory efficient
- Recursive Fibonacci: $O(2^n)$ — extremely slow; avoid for large n
- Iterative Fibonacci: $O(n)$ time, $O(1)$ space — always prefer this

Searching — Key Points

- Linear Search: works on any array; $O(n)$ worst case
- Binary Search: sorted array only; $O(\log n)$ — much faster for large n
- Binary Search works by halving the search space each step

Bubble Sort — Key Points

- $O(n^2)$ comparisons — two nested loops
- Best case $O(n)$ with early-exit optimisation (no swaps in a pass)
- Worst case: reverse-sorted input
- Stable and in-place
- After k passes, the k largest elements are in their final positions

Arrays vs Linked Lists — Key Points

- Arrays: fixed size, $O(1)$ access, $O(n)$ insert/delete at middle
- Singly Linked List: dynamic, $O(1)$ head insert, $O(n)$ access, forward only
- Doubly Linked List: bidirectional traversal, $O(1)$ deletion of known node
- Circular Linked List: last node → head; no null terminator

About your textbook

Yes — A Common-Sense Guide to Data Structures and Algorithms (Jay Wengrow) is a strong choice for T1. Its chapters on Big O, arrays, linear/binary search, and bubble/insertion/selection sort align well with your outcomes. Read Chapters 1–6 in particular. It is more visual and approachable than traditional textbooks. For linked lists, the coverage is solid enough for T1 fundamentals.

Good luck with your test, Thabiso!

ADP470S T1 | CPUT 2026